Class 11 - Solutions

April 14, 2026

#

Statistical Learning

#

Machine Learning and Econometrics

##

ECO 4971/6971

####

Class 11 — Tree-Based Methods: Exercise Solutions

Jonathan Holmes (he/him)

These exercises use the `Hitters.csv`, `Heart.csv`, and `Boston.csv` datasets.

1 In-Class Exercise #1 — Regression Trees: Terminology

Q1. In words, what is a terminal node (leaf) in a regression tree? What prediction is made for every observation that falls into the same leaf?

Answer.

A **terminal node (leaf)** is a region R_m of the predictor space where the tree makes no further splits. Every test observation that falls into region R_m — regardless of where exactly it lands within that region — receives the **same predicted value**.

For a **regression tree**, the prediction in leaf m is the **mean of the training observations** that fell into R_m:

$$\hat{y}_{R_m} = \frac{1}{|R_m|} \sum_{i:\, x_i \in R_m} y_i$$

The tree algorithm partitions the predictor space into non-overlapping rectangular regions $R_1, \ldots, R_M$ and assigns a constant prediction to each region, making the overall model **piecewise constant**.

2 In-Class Exercise #2 — Regression Trees: Interpretation

Q2. At each split, the regression tree picks a variable and a cutpoint to reduce RSS. Why might the first split be on *Years* rather than *Hits*?

Answer.

The tree greedily chooses the split that produces the **largest reduction in RSS** at each step. Formally, for a candidate variable X_j and cutpoint s, the algorithm finds:

$$\arg\min_{j,\,s} \left[\sum_{i:\,x_{ij}<s} (y_i - \bar{y}_L)^2 + \sum_{i:\,x_{ij}\geq s} (y_i - \bar{y}_R)^2 \right]$$

If *Years* provides a cleaner partition of log-salary than *Hits* — that is, if splitting on career experience creates two groups with very different average salaries — then the *Years* split reduces RSS more, so it is chosen first. In the `Hitters` data, experienced players (many years) earn substantially more on average than rookies, making *Years* a powerful first discriminator.

Q3. In a leaf, the fitted value is the mean of log(Salary). How would you convert a predicted log salary back to thousands of dollars?

Answer.

If the response was log-transformed as $y = \log(\text{Salary})$ where Salary is in thousands of dollars, then predicted salary in thousands of dollars is simply:

$$\widehat{\text{Salary}} = e^{\hat{y}}$$

For example, a leaf with mean log-salary of 6.0 corresponds to a predicted salary of $e^{6.0} \approx 403$ thousand dollars.

> **Note on retransformation bias:** the naive back-transformation $e^{\hat{y}}$ is actually the median, not the mean of Salary on the original scale. If an unbiased estimate of the mean is needed, a correction term $e^{\hat{\sigma}^2/2}$ can be applied (the smearing estimator), though for prediction purposes the naive back-transformation is standard.

Q4. If you increase `max_depth`, what usually happens to training MSE? What might happen to test MSE if depth is too large?

Answer.

`max_depth`	Training MSE	Test MSE
Increases (deeper tree)	**Decreases** (monotonically)	**Decreases, then increases**

`max_depth`	Training MSE	Test MSE
Very large	Near zero (memorises training data)	Large (overfitting)
Very small (stump)	High (underfitting)	High

Training MSE always falls or stays flat as the tree grows deeper, because more splits can only improve the fit on the training data.

Test MSE follows the classic U-shape: a shallow tree underfits (high bias); an appropriately deep tree captures the true structure; an excessively deep tree overfits noise in the training set and generalises poorly (high variance). The optimal `max_depth` is found by cross-validation.

3 In-Class Exercise #3 — Classification Trees

Q5. For classification trees, why do we not use RSS to choose splits?

Answer.

RSS measures the variance of a continuous outcome. For a **binary or categorical** response, the residual $(y_i - \bar{y})^2$ is not a meaningful measure of fit.

Instead, classification trees use **impurity measures** that quantify how mixed the class labels are in each node:

- **Gini index:** $G = \sum_k \hat{p}_{mk}(1 - \hat{p}_{mk})$ — small when one class dominates (pure node), large when classes are equally mixed.
- **Entropy (cross-entropy):** $H = -\sum_k \hat{p}_{mk} \log \hat{p}_{mk}$ — same intuition.

A split is chosen to minimise the **weighted impurity** of the two child nodes. When a node is pure (all observations belong to one class), Gini = 0 and Entropy = 0.

Q6. What does the tree predict in a leaf when the outcome is binary (e.g., AHD yes/no)?

Answer.

The tree predicts the **majority class** in the leaf: whichever class has more training observations in region R_m. If 60 of 100 training observations in leaf m have AHD = "Yes", the prediction for any new observation reaching that leaf is **AHD = Yes**.

More formally, the predicted class is $\hat{k} = \arg\max_k \hat{p}_{mk}$, where $\hat{p}_{mk}$ is the proportion of training observations in leaf m belonging to class k. The tree can also output the class probabilities $\hat{p}_{mk}$ directly, which are useful for computing ROC curves.

4 In-Class Exercise #4 — Trees vs Linear Models

Q7. Write the regression tree model as piecewise constant on regions $R_1, \ldots, R_M$. How is it different from a linear model $y = \beta_0 + \sum_j \beta_j X_j$?

Answer.

The regression tree model can be written as:

$$\hat{f}(\mathbf{x}) = \sum_{m=1}^{M} c_m \cdot \mathbf{1}(\mathbf{x} \in R_m)$$

where $c_m = \bar{y}_{R_m}$ is the mean response in leaf m and $\mathbf{1}(\cdot)$ is the indicator function.

Key differences from a linear model:

	Linear model	**Regression tree**
Functional form	Globally linear: $\hat{f}(\mathbf{x}) = \beta_0 + \sum_j \beta_j X_j$	Locally constant: different constant in each region
Continuity	Smooth, continuous	Discontinuous at split boundaries
Interactions	Only if explicitly specified ($X_j X_k$ terms)	Implicit through sequential splits
Interpretability	Coefficients have direct marginal-effect interpretation	Decision rules are intuitive but non-parametric
Extrapolation	Extrapolates to unseen regions	Cannot extrapolate beyond training data range

---

Q8. Give one setting where you might prefer a linear model, and one where a tree might be more attractive.

Answer.

Prefer linear model: predicting a worker's wage from education and experience. Economic theory suggests a smooth, approximately linear relationship (Mincer earnings equation). A linear model is interpretable, allows marginal-effect inference, and is efficient when the assumption holds.

Prefer a tree: detecting fraud in financial transactions, where the risk profile depends on complex, non-linear interactions (e.g., transaction amount matters a lot *only* when the account is young *and* the location is foreign). A tree naturally captures such interactions without requiring them to be pre-specified, and the resulting decision rules are easy to explain to a compliance team.

5 In-Class Exercise #5 — Bagging

Q9. What are out-of-bag (OOB) observations used for when you fit many bagged trees?

Answer.

When we fit a bagged ensemble, each tree is trained on a **bootstrap sample** — a random sample of n observations drawn *with replacement* from the training set. On average, each bootstrap sample contains about **63.2%** of the distinct training observations; the remaining ~36.8% are **out-of-bag (OOB)** for that tree.

Because an OOB observation was not used to train the tree that omitted it, it can be used as a **natural validation set**:

1. For each training observation i, collect the predictions from **all trees for which i was OOB**.
2. Average these predictions to obtain an OOB prediction $\hat{y}_i^{\text{OOB}}$.
3. Compute **OOB MSE** $= \frac{1}{n}\sum_i (y_i - \hat{y}_i^{\text{OOB}})^2$.

The OOB error is a valid estimate of test error **without needing a separate validation set or cross-validation** — the held-out structure is built into the bootstrap. OOB error can also be used to tune the number of trees and (in Random Forests) to compute variable importance.

6 In-Class Exercise #6 — Gradient Boosting

Q10. In the Boston case study you tune learning rate λ and tree depth. Holding other settings fixed, what is the usual bias–variance tradeoff when you increase depth?

Answer.

In boosting, the **tree depth** (or equivalently, `max_depth` / number of leaves) controls the complexity of each individual tree (weak learner):

`max_depth`	Bias	Variance
Shallow (depth 1–2, stumps)	**High** — each tree can only model a simple main effect; the ensemble may underfit even with many trees	**Low** — each tree is simple, the ensemble averages out variance
Deep (depth 5+)	**Low** — each tree can capture complex interactions	**High** — each tree overfits its residuals; the ensemble becomes sensitive to training data

Shallow trees (depth 1–2) are common in boosting because the **sequential nature of boosting** allows the ensemble to build up complexity gradually across many trees, compensating for each tree's limited depth. Using deeper trees can speed convergence but risks overfitting unless the learning rate λ and number of trees are carefully tuned together.

Interaction with learning rate: a small λ with many shallow trees is usually the most stable configuration; increasing depth while keeping λ small is safer than increasing depth while keeping λ large.

Q11 ***(Optional / take-home)*****. Re-run the case study with a different `random_state` on `train_test_split`. How much do test MSE and R^2 move? What does that say about the variance of a single tree or tuned ensemble?**

Answer.

Changing `random_state` produces a different 80/20 train–test split. For **a single decision tree**, test MSE and R^2 can shift substantially (sometimes 10–20%) because a single tree has high variance — the specific observations in the training set heavily influence which splits are chosen.

For a **tuned gradient boosting ensemble** (or Random Forest), the test metrics should be much more stable across random seeds, because:

1. Averaging over many trees reduces the variance of individual tree predictions.
2. The ensemble is less sensitive to any particular training observation.

Large swings in test performance across seeds indicate **high variance** — a signal that the model is overfitting or that the dataset is too small. Robust models should show similar (though not identical) test performance regardless of the random seed. This is why reporting performance from a single train–test split is risky; cross-validation provides a more reliable estimate.